Progetto “Gioco della briscola multiplayer”

Relazione finale

Ingegneria del software : Tecnologie Internet
Prof. Michele Amoretti

di Matteo Ibridi, Pietro Marossa, Nicola Ferraroni

- Introduzione

  Il presente documento illustra le fasi di organizzazione, progettazione e implementazione di un applicazione web dedicata al gioco della Briscola. Oltre alla semplice struttura del gioco, questo progetto pone attenzione all'utilizzo di tecnologie Peer to Peer e alla loro implementazione in situazioni di gioco realistiche. L'idea del progetto è quella di fornire un servizio multiplayer che permetta a utenti di affrontarsi in tempo reale, memorizzando storici e dati per ognuno di essi.

- Tecnologie utilizzate
  - Sviluppo Frontend
    - Html & Css : per struttura di base come tavolo da gioco, carte e schermate in app;
    - Javascript : per gestione dinamica delle operazioni di gioco e iterazioni utenti;
  - Sviluppo Backend e Gestione rete:
    - Express.js (con Nodejs) : per gestione api, necessarie per inizializzare connessione e avvio sessioni di gioco, con relativo contatto tra gli utenti;
    - Json : per trasmissione dei dati all'interno della rete, con aggiornamenti costanti per ogni azione di gioco;

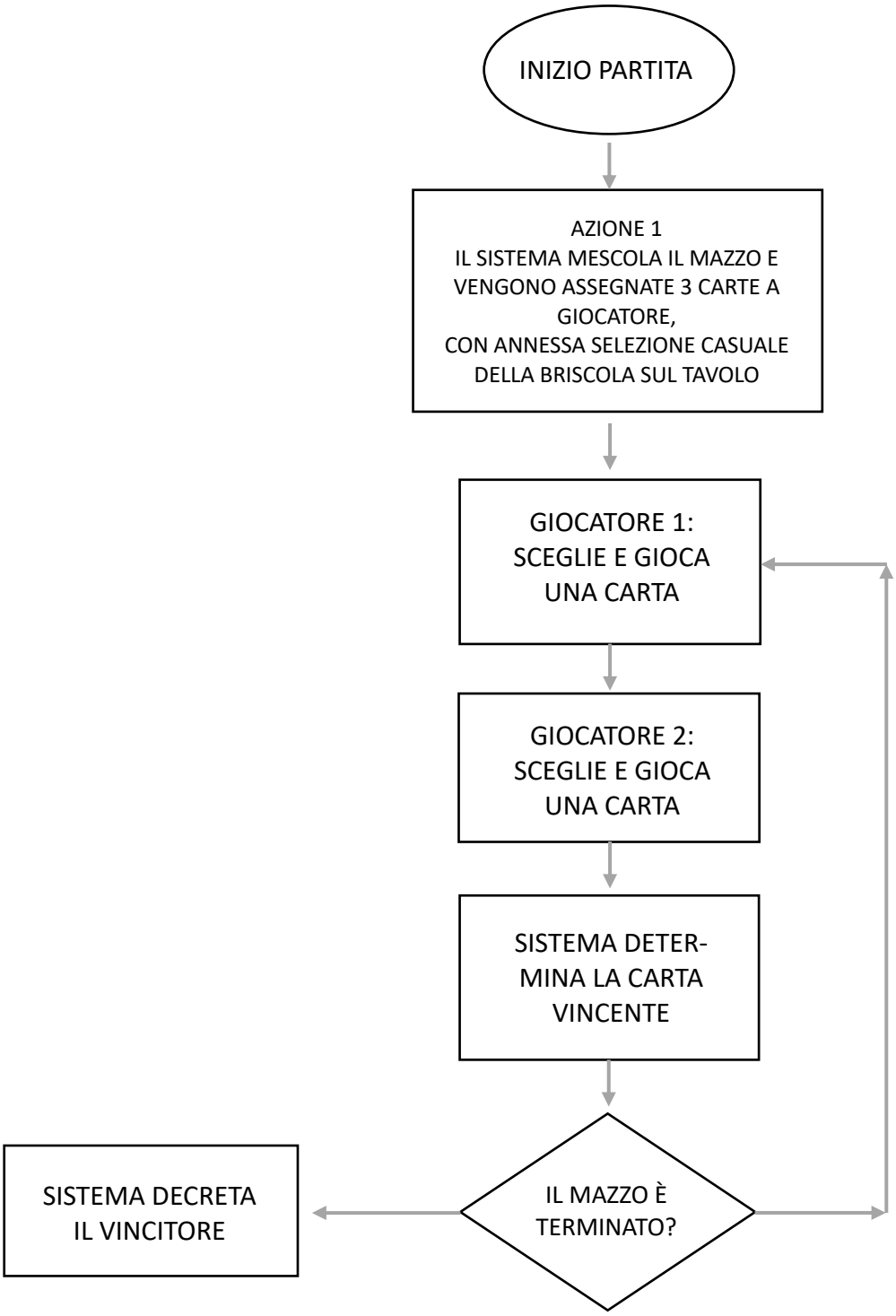
- Specifica dei requisiti

  - Requisiti funzionali
    - L'utente dà avvio alla sessione inserendo il codice dell'avversario e connettendosi al server si inizializza un incontro.
    - Il funzionamento segue le regole classiche della briscola a 2 giocatori, dando priorità a semi e carichi e riconoscendo la carta vincente in ogni mano.
    - A fine incontro, il sistema calcola i punti totali realizzati dai giocatori e decreta il vincitore.
    - Successivamente si aggiorna lo storico degli utenti e si ritorna alla schermata principale.

  - Requisiti non funzionali

- La comunicazione avviene tramite P2P,
- Le interfacce principali sono 3, dedicate al menù principale, alla schermata di gioco in esecuzione e alla schermata dello storico dedicata.
- Il sistema prevede che ogni utente veda le proprie carte e non quelle dell'avversario; solo alla fine si rivelerà il punteggio e si decreterà il vincitore dopo il conteggio automatico.

- Progettazione e architettura
  - Diagramma di flusso da mano di gioco a decreto del vincitore



- Scelte su architettura
    - Per questo progetto, si è scelto di usare un architettura P2P, creando un ambiente in cui i due utenti intereagiscono tra loro in modo diretto e contemporaneo.
    - Si è scelto di utilizzare un file Json come database di sviluppo, per tenere seguito ai cambiamenti dei dati durante l'utilizzo.
    - Per le funzionalità dell'applicazione, si è scelto di usare varie api che potessero gestire queste necessità, quali la creazione di utenti personali, la lettura e l'aggiornamento delle statistiche di ognuno di essi, infine la possibilità di modificare o rimuovere utenti e statistiche allocate.

- Implementazione
    - Struttura codice
        - Interfaccia utente: il frontend dell'applicazione è stato creato utilizzando React con deploy sulla piattaforma Netlify. Questa parte si dedica a tutta la parte grafica, quali schermate di gioco, tavolo, carte visualizzate e le varie iterazioni degli utenti. L'accesso del frontend ai dati lato server non è diretto, ma gestito tramite richieste http.
        - Lato Client-Server: si è scelto di utilizzare un architettura API REST per la comunicazione tra frontend e backend, con l'ultimo implementato tramite Node.js con framework express. La logica di base viene gestita dal file server.js, mentre si ha stats.json come database per il salvataggio di informazioni. Le varie richieste sono gestite da fetch() che gestirà l'accesso ai file.
        - Rete P2P : la fase di connessione tra i due giocatori è gestita in 2 fasi: nella prima avviene il contatto primario tra i due utenti, dei quali uno contatta il Peer Server e fa richiesta per l'inizializzazione di una partita, ottenendo un ID univoco. Successivamente tramite peer.connect() il secondo utente instaura il collegamento utilizzando l'ID fornito. Nella seconda fase, la comunicazione tra i due utenti è diretta. Infatti, una volta stabilito il matchmaking, le mosse viaggiano in modalità browser-browser, con l'utilizzo eventuale di Turn in caso di connessioni bloccate da firewall o in presenza di Nat.

- Conclusione
  La sfida principale di questo progetto è stata creare una certa architettura di gioco, che permettesse una visione sempre chiara e coerente

del tavolo e del mazzo nello svolgimento della partita. Il tutto sfruttando le tecnologie illustrate nel corso e seguendo la tradizione di uno dei giochi di carte più famosi nel nostro paese.